かなえ Discord Bot — システムアーキテクチャ

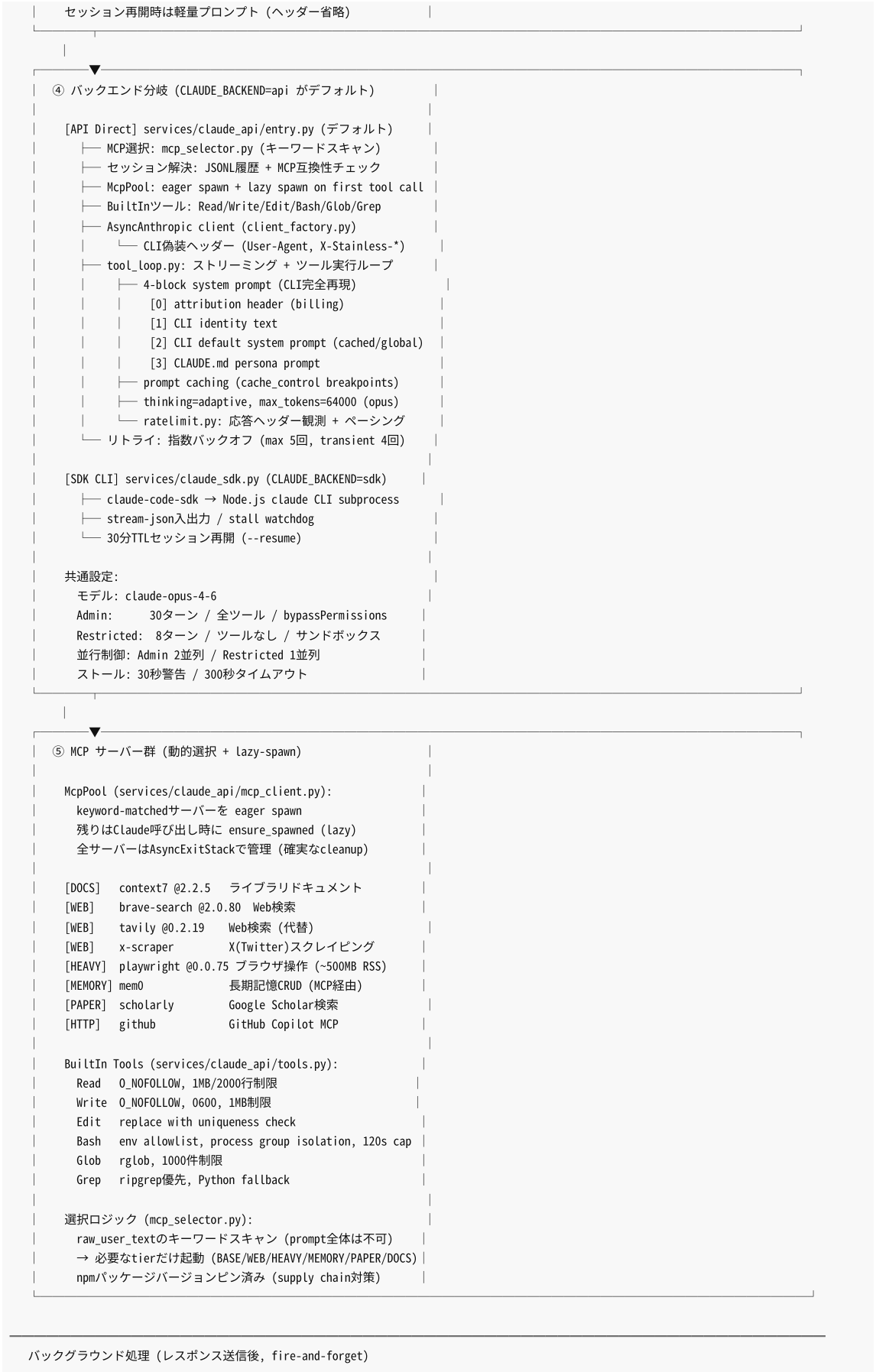
概要

Claude (Anthropic) をバックエンドとするAI Discord Bot。単純なAPI呼び出しではなく、長期記憶・感情エンジン・セッション持続・動的ツール選択を備えた自律的エージェントとして設計されている。2GB RAMのUbuntuサーバー1台で稼働する。

---

1. アーキテクチャ全体像





バックグラウンド処理（レスポンス送信後，fire-and-forget）

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│  ⑥ 長期記憶 (Mem0) — services/mem0/                                                                          │
│                                                                                                              │
│  ┌── Ingest Pipeline (ingest.py) ──────────────────────────────────────────┐                                 │
│  │  会話テキスト (user + assistant)                                        │                                 │
│  │    → extract.py: Claude Sonnet fact抽出                                 │                                 │
│  │      重要度1-5 / category / subcategory / tags /                        │                                 │
│  │      entities (名前, 型, 別名) を構造化抽出                             │                                 │
│  │    → importance >= 2 ゲーティング (低品質factを除外)                    │                                 │
│  │    → intra-batch dedup (同一バッチ内テキスト重複除去)                   │                                 │
│  │    → dedup.py: 2段階near-dup排除                                        │                                 │
│  │      Stage 1: ChromaDB ベクトル類似度検索                               │                                 │
│  │      Stage 2: Haiku LLM意味的重複判定                                   │                                 │
│  │    → reinforce.py: 重複候補のmention_count++                            │                                 │
│  │      (Hebbian reinforcement: 繰り返し言及 = 重要)                       │                                 │
│  │    → graph.py: entity upsert + fact↔entity edge追加                     │                                 │
│  │    → ChromaDB: infer=False raw insert (metadata付き)                    │                                 │
│  └──────────────────────────────────────────────────────────────────────────┘                                 │
│                                                                                                              │
│  ┌── Search Pipeline (search.py) ──────────────────────────────────────────┐                                 │
│  │  クエリテキスト                                                         │                                 │
│  │    → graph.py: alias展開 (例: "クラナド"→"CLANNAD")                     │                                 │
│  │    → Haiku query rewrite (正準キーワード3-5個)                          │                                 │
│  │    → ChromaDB ベクトル検索 (multi-query, dedup merge)                   │                                 │
│  │    → graph.py: 1-hop graph展開 (関連entity経由の                        │                                 │
│  │                sibling facts を候補追加)                                │                                 │
│  │    → reinforce.py: reinforcement boost                                  │                                 │
│  │      (高mention_count factの距離スコアを縮小)                           │                                 │
│  │    → Haiku LLM rerank (top-5選出)                                       │                                 │
│  └──────────────────────────────────────────────────────────────────────────┘                                 │
│                                                                                                              │
│  双方向記憶:                                                                                                │
│    ├── ユーザー記憶 (discord_user_id namespace)                                                              │
│    │    extractorは先輩の発言からfactを抽出                                                                  │
│    └── かなえ自己記憶 (bot_user_id namespace)                                                                │
│         extractorはかなえ(assistant)の発言からfact抽出                                                       │
│                                                                                                              │
│  Entity Graph (graph.py, SQLite):                                                                            │
│    canonical_name / type / user_id / aliases                                                                 │
│    fact_entity edges (多対多)                                                                                │
│    cross-language alias解決 + 1-hop related fact検索                                                         │
│                                                                                                              │
│  メタデータ: 8カテゴリ taxonomy + importance(1-5) + tier(0-3)                                                │
│  メンテナンス: 週次compaction (日曜9時cron)                                                                  │
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────┘


┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│  ⑦ 感情エンジン (services/emotion/)                                                                          │
│                                                                                                              │
│  ┌── Persistent Haiku Daemon (persistent_session.py) ──────────┐                                            │
│  │  claude --print --input-format stream-json                   │                                            │
│  │         --output-format stream-json --model haiku            │                                            │
│  │    ├── bot起動時に1回だけspawn, プロセスライフタイム維持     │                                            │
│  │    ├── セッション内記憶でキャラクター一貫性を維持            │                                            │
│  │    ├── asyncio.Lock でフィードバックと沈黙ティックを直列化   │                                            │
│  │    └── crash/hang時: query_timeout(60s) → kill → auto-restart│                                           │
│  └──────────────────────────────────────────────────────────────┘                                            │
│                                                                                                              │
│  状態ベクトル (6次元, kanae_emotion.json):                                                                   │
│    valence(-1~+1), arousal(0~1), loneliness(0~1),                                                            │
│    affection(0~1), anxiety(0~1), curiosity(0~1)                                                              │
│    + derived_labels (LLM生成, 最大3個)                                                                       │
│    + labels (6軸それぞれの日本語ラベル)                                                                      │
│    + emotion_journal (ring buffer, 最大25件)                                                                 │
│    + last_interaction (timestamp, quality, initiated_by)                                                     │
│                                                                                                              │
│  遷移トリガー:                                                                                               │
│    ├── 会話フィードバック (feedback.py)                                                                      │
```

```
│     │    on_message完了後 fire-and-forget                        │
│     │    daemon.query() → llm_transition_via_daemon()           │
│     │    → validate_and_apply() → save_under_lock()             │
│     │                                                           │
│     ├── 沈黙ティック (_silence_tick_loop, bot.py内)              │
│     │    30分間隔, in-process永続ループ                          │
│     │    会話中はスキップ (is_conversation_active flag)          │
│     │    → daemon.query(mode="silence")                         │
│     │                                                           │
│     └── 自発連絡 (self_initiate.py)                             │
│         沈黙ティック完了後に実行                                  │
│         telos.py: 感情状態 → 行動動機を生成                      │
│           loneliness_overflow / share_joy / anxiety_relief /    │
│           affection_expression / topic_driven / reconciliation  │
│         動機があれば → 1-shot Claude CLI でメッセージ生成        │
│         → bot.get_channel().send() でDiscord投稿                 │
│         cooldown: 1時間 (sentinel file)                          │
│                                                                 │
│   出力:                                                         │
│     ├── <kanae_state> ブロック (毎ターンのプロンプト注入)        │
│     ├── <kanae_daily> ブロック (生活シミュレーション)            │
│     └── 1時間サマリーDM (daily_summary_dm.py → admin宛)          │
│                                                                 │
│   日常ログ (daily.py):                                          │
│     kanae_daily.json: 96エントリring buffer (~24h @15min)       │
│     kanae_daily_summary.txt: LLM生成ランニングサマリー          │
│     kanae-schedulerサービス (外部) が15分毎にエントリ追加        │
└─────────────────────────────────────────────────────────────────┘

═══════════════════════════════════════════════════════════════════
  認証 / セキュリティ層
═══════════════════════════════════════════════════════════════════

┌─────────────────────────────────────────────────────────────────┐
│   ⑧ OAuth管理 (services/oauth_manager.py)                       │
│                                                                 │
│     Claude Code OAuth (platform.claude.com):                    │
│       ├── ~/.claude/.credentials.json からトークン読込          │
│       ├── 有効期限5分前に自動リフレッシュ                       │
│       ├── Anthropic SDK monkey-patch (sync Anthropic client)    │
│       ├── AsyncAnthropic直接注入 (async client)                 │
│       └── Subscriber session priming:                           │
│           /api/oauth/profile + /api/oauth/claude_cli/roles      │
│           を初回呼び出し時にGET (subscriber-tier認識のため)     │
│                                                                 │
│     Subscriber-tier identification (client_factory.py):         │
│       ├── User-Agent: "claude-cli/2.1.92 (external, sdk-cli)"   │
│       ├── X-Stainless-*: Node.js CLIのヘッダーを完全再現        │
│       ├── x-anthropic-billing-header: version+suffix hash       │
│       ├── metadata.user_id: device_id+account_uuid+session_id   │
│       └── 4-block system prompt (CLI実行時の構造を再現)         │
│                                                                 │
│     Rate-limit pacing (ratelimit.py):                           │
│       ├── anthropic-ratelimit-unified-* ヘッダーを毎回観測      │
│       ├── 5h/7d utilization, fallback-percentage を追跡         │
│       ├── allowed_warning時: 3秒gap / throttle時: 10秒gap       │
│       └── fallback>=99%時のみ完全ブロック (overage許容)         │
└─────────────────────────────────────────────────────────────────┘

┌─────────────────────────────────────────────────────────────────┐
│   ⑨ セキュリティモデル                                          │
│                                                                 │
│     Admin (ADMIN_USER_IDS):                                     │
│       ├── 全BuiltInツール + 全MCPツール                         │
│       ├── bypassPermissions                                     │
│       ├── MemO読み書き (ユーザー記憶 + かなえ自己記憶)          │
│       ├── 感情状態の読み書き                                    │
│       └── OAuth subscriber token使用                            │
│                                                                 │
│     Restricted (全その他ユーザー):                              │
```

```
│       ├── ツール一切なし (allowed_tools=[], deny list)          │
│       ├── MCP接続なし                                          │
│       ├── CWDをtmpdir隔離 (/tmp/discord_bot_attachments)       │
│       ├── 専用APIキー (BOT_RESTRICTED_ANTHROPIC_API_KEY)       │
│       └── MemO書き込み不可 (poisoning防止)                     │
│                                                               │
│    プロンプトインジェクション対策 (sentinel.py):               │
│       ├── NFKC Unicode正規化 + ゼロ幅文字除去                  │
│       ├── sentinel regex (XMLタグ偽装検出・除去)               │
│       ├── power phrase regex (admin/root権限詐称除去)          │
│       ├── <untrusted_user_input> XMLラッピング                 │
│       ├── 非admin: admin発言のinline展開を拒否                 │
│       └── MemO: 非adminの記憶書き込みを遮断 (C-7 poisoning)    │
│                                                               │
│    ファイルセキュリティ:                                       │
│       ├── 全state/historyファイル: mode 0o600                  │
│       ├── atomic_write_json: mkstemp → fchmod → rename         │
│       ├── Read/Write: O_NOFOLLOW (symlink traversal防止)       │
│       ├── Bash: env allowlist (PATH,HOME,USER等のみ)           │
│       └── 画像添付: ALLOWED_IMAGE_DIRS ホワイトリスト          │
└───────────────────────────────────────────────────────────────┘

═══════════════════════════════════════════════════════════════
  出力処理
═══════════════════════════════════════════════════════════════

┌───────────────────────────────────────────────────────────────┐
│  ⑩ レスポンスパイプライン                                      │
│                                                               │
│    Claude応答テキスト                                          │
│      → strip_numeric_inline_code (数値の`バッククォート`除去)  │
│      → normalize_image_filenames (相対パス→絶対パス)           │
│      → parse_segments (text / table / math を分離)            │
│      → Markdownテーブル → PNG (Puppeteer, table_renderer.py)  │
│      → LaTeX数式 → PNG (KaTeX, math_renderer.py)              │
│      → split_message (2000文字制限で自然分割)                  │
│      → extract_image_paths → discord.File添付 (max 10枚)      │
│      → extract_file_paths → 任意ファイル添付 (admin only)     │
│      → 8000文字超 & テーブル/数式なし → .md ファイル送信       │
│      → Discord送信 (message.reply)                            │
└───────────────────────────────────────────────────────────────┘
```

---

2. データフロー (1リクエスト)

```
User → Discord WS / Socket.IO → on_message / send_message
  ├── Gate check (bot? DM? mention? rate limit? quota?)
  ├── ☒ → reject with message
  └── ☒ → acquire channel semaphore (1 req/channel, LRU 500)
      │
      ├──[並列] reply_context, channel_history, link_resolve,
      │         cross_thread_ctx, mem0_search(smart), kanae_self_search,
      │         image_download, text_attachments, emotion_load
      │
      ▼
    Prompt Assembly (bot.py)
      │  レイヤー: emotion → daily → self_mem → user_mem → header → untrusted
      │  セッション再開時: 軽量プロンプト (untrusted blockのみ)
      ▼
    MCP Server Selection (mcp_selector.py)
      │  raw_user_textキーワード → 必要なtierだけ起動
      │  → session resume判定 (MCP互換性チェック)
      ▼
    Claude API Direct (entry.py → tool_loop.py)
      │  AsyncAnthropic client (CLI偽装ヘッダー)
      │  rate-limit pacing → streaming → status callback → Discord更新
      │   ├── thinking block → "☒ thinking..."
      │   ├── text block → "☒ 応答を生成中"
      │   ├── tool_use → can_use_tool gate → execute → tool_result
```

```
        │   │             MCP tool: McpPool.call_tool (lazy spawn)
        │   │             BuiltIn: execute_builtin_tool
        │   └── end_turn / max_turns → 完了
        ▼
    Response Processing
      ├── テキスト後処理 (sanitize, render tables/math, split)
      ├── Discord送信 (reply + file attachments)
      │
      └──[fire-and-forget background tasks]
           ├── Mem0 ingest: ユーザーfact抽出 (Sonnet)
           ├── Mem0 ingest: かなえ自己fact抽出 (Sonnet)
           ├── Emotion feedback: daemon.query(conversation) → state update
           ├── JSONL history append (user + assistant turns)
           └── Session info persistence (sessions.json)
```

---

3. プロセスアーキテクチャ

```
main_async.py (PID 1, pidfile lock)
  │
  ├── asyncio.gather:
  │     ├── bot.start(DISCORD_TOKEN)          Discord Gateway接続
  │     └── web_api.start(port=3002)          Socket.IO + aiohttp
  │
  ├── on_ready hooks:
  │     ├── emotion_daemon_start              Haiku常駐プロセス spawn
  │     ├── emotion_silence_tick              30分ループ (in-process)
  │     ├── hourly_summary_dm                 1時間ループ (admin DM)
  │     ├── vscode_sync daemon                VSCode Claude Code → Discord
  │     └── cleanup_stale_attachments         tmpdir掃除 (1回)
  │
  └── graceful shutdown (SIGTERM/SIGINT):
       ├── emotion daemon stop
       ├── kill_orphan_claude_cli/mcp
       ├── bot.close (Discord disconnect)
       └── aiohttp cleanup
```

---

4. Web API (Socket.IO)

```
Socket.IO Server (aiohttp, port 3002)
  │
  ├── 認証: Next.js /api/session/verify → session cache (300s TTL)
  │         ADMIN_USER_IDS による admin 判定 (server-side, not trusted from Next.js)
  │
  ├── イベント:
  │    connect        セッショントークン認証
  │    join_channel   チャンネル権限チェック → room join
  │    get_history    JSONL履歴読込 (Mem0ブロック除去)
  │    send_message   → PromptBuilder → call_with_tools → stream status → response
  │    disconnect     in-flight task cancel
  │
  ├── Rate limiting:  Discord側と同じロジック
  ├── Channel perms:  Discord API経由のguild member権限チェック
  │
  └── /health         subsystem snapshot (discord, oauth, mem0)
    /internal/invalidate_session  (localhost only)
```

---

5. 技術スタック

層	技術
ランタイム	Python 3.12 / asyncio
Discord	discord.py (commands.Bot)
Web UI	Socket.IO (aiohttp) + Next.js フロント
AI (メイン)	Claude opus-4-6 (OAuth subscriber認証)

AI (fact抽出)	Claude Sonnet (extract.py)
AI (感情/rewrite/rerank)	Claude Haiku 4.5
Embedding	OpenAI text-embedding-3-small (Mem0内部)
ベクトルDB	ChromaDB (Mem0)
Entity Graph	SQLite (services/mem0/graph.py)
MCP	8サーバー (context7, playwright, brave, x-scraper, scholarly, tavily, github, mem0)
レンダリング	Puppeteer (テーブル→PNG), KaTeX (数式→PNG)
インフラ	Ubuntu / 2GB RAM / systemd
ログ	structured logging (correlation ID) + RotatingFileHandler

---

6. 設計上の特徴

6.1 CLI偽装によるSubscriber-tier識別

Anthropic Messages APIはClaude Code CLIからの呼び出しかどうかでrate-limit tierが異なる。Python SDKのデフォルトヘッダーでは一般OAuthバケットに分類され、subscriber quotaを活用できない。client_factory.pyがNode.js CLIのUser-Agent, X-Stainless-*, billing header, 4-block system prompt構造を完全に再現し、subscriber tierでの呼び出しを実現している。

6.2 動的MCPサーバー選択 + Lazy Spawn

全8サーバーを常時起動するとメモリが足りない（playwright単体~500MB）。mcp_selector.pyがユーザーテキストをキーワードスキャンし、必要なサーバーだけeager spawn。さらにMcpPoolがlazy spawn対応しており、Claudeが予期しないMCPツールを呼んだ場合もその時点で自動起動する。セッション再開時はMCPの差分を検出し、不足があれば新規セッションを開始。

6.3 二重権限モデル

Admin（開発者）とRestricted（一般ユーザー）で完全に異なるClaude設定。Restrictedはツール・MCP・ファイルシステムへのアクセスが一切ない。専用APIキーで課金分離。プロンプトインジェクション対策はNFKC正規化 + ゼロ幅文字除去 + sentinel regex + XMLタグ分離の多層防御。

6.4 LLMベース感情エンジン

代数的な状態遷移（`if X then Y`）ではなく、Claude Haiku常駐デーモンが会話内容を分析して感情状態を遷移させる。デーモンはプロセスライフタイム中セッションを維持し、過去の感情文脈を覚えている。会話フィードバック + 30分沈黙ティック + telos駆動の自発連絡 + 1時間サマリーDMで、会話がない時間帯にも感情が動く。

6.5 スマート記憶 (Entity Graph + Hebbian Reinforcement)

単純なベクトル検索ではなく5段パイプライン: alias展開(graph) → query rewrite(Haiku) → multi-query vector search → 1-hop graph展開 → LLM rerank。書き込みはSonnet fact抽出 + importance gating + 2段階dedup + Hebbian reinforcement（重複候補のmention_count加算で重要度が自然上昇）。Entity Graphで日英cross-language検索も対応。

6.6 セッション持続 (JSONL履歴ベース)

Anthropic Messages APIはステートレスなので、JSONL transcriptを毎ターン保存し、次のターンで全履歴をreplay。30分TTL内の同一チャンネル会話はsynthetic session IDで管理。再開時はプロンプトを軽量化（ヘッダー・コンテキスト省略）してトークン節約。MCP互換性チェックで、必要なMCPサーバーが変わった場合は自動的に新規セッション。

6.7 Rate-limit Pacing

Anthropic APIの`anthropic-ratelimit-unified-*`レスポンスヘッダーを毎回観測し、5h/7d utilization, fallback-percentageを追跡。warning状態で3秒、throttle状態で10秒のペーシングギャップを自動挿入。fallback>=99%の場合のみリクエストを完全ブロック（overageパスを許容するため、rejected状態でもブロックしない）。

---

7. ファイル構成

```
discord-bot/
├── bot.py                  # Discordイベントハンドラ / メッセージルーティング
├── config.py               # 設定・環境変数・MCP定義・システムプロンプト
├── main_async.py           # エントリポイント (Discord + WebAPI 同時起動)
├── main.py                 # 旧エントリポイント (Discord only)
├── mcp_mem0_server.py      # Mem0 MCPサーバーラッパー
├── web_api.py              # Socket.IO Web API (チャット + ヘルスチェック)
│
├── services/
```

```
│   ├── claude_sdk.py         # Claude Code SDK (CLIサブプロセス, fallback)
│   │                         CLAUDE_BACKEND=api時はdispatcher経由で
│   │                         services/claude_api/に委譲
│   │
│   ├── claude_api/           # Claude API直接呼び出し (デフォルトバックエンド)
│   │   ├── __init__.py       #   re-export hub + call_direct (tool不使用パス)
│   │   ├── entry.py          #   call_with_tools: セッション/MCP/リトライ統合
│   │   ├── client.py         #   定数: CC_VERSION, MODEL_BETAS, attribution
│   │   ├── client_factory.py #   AsyncAnthropic構築 (CLI偽装ヘッダー)
│   │   ├── tool_loop.py      #   run_agent_loop: ストリーミング+ツール実行
│   │   ├── tools.py          #   BuiltIn Tools (Read/Write/Edit/Bash/Glob/Grep)
│   │   ├── mcp_client.py     #   McpPool + MCP subprocess管理
│   │   ├── restricted.py     #   非admin: deny list + ツール制限
│   │   ├── session_store.py  #   session TTL管理 (in-memory timestamps)
│   │   ├── retry.py          #   リトライ予算 + 指数バックオフ
│   │   ├── errors.py         #   エラー分類 (rate_limit/auth/context等)
│   │   ├── stall.py          #   StallWatchdog (ストリーム停止検知)
│   │   ├── status.py         #   ステータスコールバック + ツールヒント
│   │   ├── ratelimit.py      #   subscriber-tier rate-limit pacing
│   │   └── image_processor.py#   画像ブロック抽出 + base64変換
│   │
│   ├── oauth_manager.py      # OAuth管理 / SDKパッチ / subscriber priming
│   ├── prompt_builder.py     # プロンプト組立 (Web API用、bot.pyも一部使用)
│   ├── mcp_selector.py       # MCPサーバー動的選択 (キーワードトリガー)
│   ├── session.py            # セッション永続化 + JSONL履歴管理
│   ├── rate_limit.py         # 共有レート制限モジュール
│   ├── sentinel.py           # プロンプトインジェクション防御 (NFKC + regex)
│   ├── correlation.py        # リクエストID (contextvars)
│   ├── structured_log.py     # JSON structured logging
│   ├── health.py             # /health サブシステム状態集約
│   ├── mem0_service.py       # Mem0 facade (search/add re-export)
│   │
│   ├── mem0/                 # 長期記憶サブシステム
│   │   ├── client.py         #   ChromaDB + OpenAI embedding初期化
│   │   ├── search.py         #   smart search (rewrite + graph + rerank)
│   │   ├── ingest.py         #   書き込みパイプライン (gate + dedup + graph)
│   │   ├── extract.py        #   Sonnet fact抽出 (user/kanae dual extractor)
│   │   ├── dedup.py          #   2段階near-dup排除 (embedding + LLM)
│   │   ├── reinforce.py      #   Hebbian reinforcement (mention_count++)
│   │   ├── graph.py          #   Entity Graph (SQLite, alias, 1-hop)
│   │   ├── format.py         #   記憶テキストのプロンプト整形
│   │   └── cli_compat.py     #   Haiku呼び出しラッパー (attribution付き)
│   │
│   ├── emotion/              # 感情エンジンサブシステム
│   │   ├── persistent_session.py  # Haiku常駐デーモン (stream-json CLI)
│   │   ├── llm_transition.py      # LLM感情遷移 (conversation/silence)
│   │   ├── feedback.py            # 会話分析 → daemon → 状態更新
│   │   ├── self_initiate.py       # telos駆動の自発連絡
│   │   ├── telos.py               # 行動動機生成 (純粋関数)
│   │   ├── daily.py               # 日常ログ ring buffer
│   │   ├── daily_summary_dm.py    # 1時間サマリーDM
│   │   ├── state.py               # EmotionState dataclass + load/save
│   │   ├── format.py              # <kanae_state>ブロック整形
│   │   ├── validator.py           # LLM出力検証 + state適用
│   │   ├── session.py             # daemon session ID管理 (UUID永続化)
│   │   ├── runtime.py             # EMOTION_CTX (contextvars, daemon共有)
│   │   ├── lock.py                # flock排他制御
│   │   ├── params.py              # 閾値・境界値定数
│   │   ├── analyzer.py            # (legacy: 会話分析)
│   │   ├── transition.py          # (legacy: 代数的遷移)
│   │   └── cli.py                 # Claude CLIラッパー (subprocess)
│   │
│   └── vscode_sync/          # VSCode Claude Code → Discord同期
│       ├── daemon.py         #   JSONL tailerメインループ
│       ├── tailer.py         #   ファイル末尾追跡
│       ├── parser.py         #   Claude Code JSONL解析
│       ├── aggregator.py     #   メッセージ集約 (flush interval)
│       ├── discord_post.py   #   Discord thread投稿
│       └── redactor.py       #   機密情報マスキング
│
```

```
├── utils/
│   ├── attachments.py        # 画像/ファイルダウンロード・添付処理
│   ├── discord_helpers.py    # メッセージ分割/リンク解決/履歴取得
│   ├── table_renderer.py     # Markdownテーブル → PNG (Puppeteer)
│   ├── math_renderer.py      # LaTeX数式 → PNG (KaTeX)
│   ├── markdown_parser.py    # セグメント解析 (text/table/math)
│   ├── response_sanitizer.py # 数値インラインコード除去
│   ├── atomic_write.py       # atomic JSON/text write (mkstemp)
│   ├── browser_pool.py       # Puppeteer ブラウザプール
│   ├── render_semaphore.py   # レンダリング並行制御
│   ├── image_validator.py    # 画像パス検証
│   └── cache.py              # TTL付きキャッシュ
│
├── scripts/
│   ├── weekly_mem0_compact.py    # 週次記憶圧縮 (cron)
│   ├── ingest_channel_history.py # チャンネル履歴一括取込
│   ├── kanae_daily.py            # 日常エントリ生成 (kanae-scheduler呼出)
│   ├── emotion_feedback_cli.py   # CLI感情フィードバック
│   └── ...                       # backfill, audit, cleanup scripts
│
└── tests/                    # テスト (~80ファイル)
```

---

8. 外部サービス連携

サーバー上の関連サービス	
kanae-scheduler	日常ログ生成 (15分cron)
kanae-contact	連絡先管理
kanae-voice-mcp	音声入力MCP
news-curator	ニュースキュレーション
happy-server	Next.js Web UI バックエンド
english-app	英語学習アプリ
scholarly-mcp	Google Scholar MCP
free-code / CLI	開発ツール
/opt/timetable	時間割API
cloudflared	Cloudflare Tunnel

---

9. 運用情報

- **サーバー**: Ubuntu / 2GB RAM / systemd managed
- **ログ**: structured JSON (correlation ID付き) + RotatingFileHandler (10MB×3)
- **ヘルスチェック**: `/health` エンドポイント (discord, oauth, mem0 subsystem check)
- **OAuth監視**: リフレッシュ失敗で自動リトライ、3連続失敗でアラート
- **孤児プロセス回収**: `kill_orphan_claude_cli()` + `kill_orphan_mcp_processes()` (psutil)
- **PID lock**: `.state/bot.pid` (fcntl.LOCK_EX) で二重起動防止
- **週次メンテナンス**: 日曜9時 mem0 compaction cron
- **ToolSearch無効化**: `ENABLE_TOOL_SEARCH=false` (deferred tool空結果バグ回避)